

NAME: K. HARSHIKA

REG.NO:192473020

COURSE CODE: CSA0609

ASSESSMENT 3

Debugging Convex Hull Logic (Collinear Case Mishandling)

The following brute force Convex Hull edge validation logic fails in cases where all remaining points are collinear with a candidate edge. Carefully analyze how collinear points should be handled and explain why the current condition may reject valid hull edges or include incorrect ones. Debug the logic to correctly distinguish between boundary edges and invalid interior segments. Optimize the implementation by reducing repeated orientation checks if possible. Analyze the time complexity of the corrected brute force solution and present the improved code clearly.

```
def hull_edge(p1, p2, points):  
    for p in points:  
        val = orientation(p1, p2, p)  
        if val == 0:  
            return False    # ERROR  
        if val < 0:  
            return False  
    return True
```

ANSWER:

First Error:

```
if val == 0:  
    return False
```

What is the error?

The program immediately returns False whenever it finds a point that is collinear with the candidate edge (p1, p2). However, in the Convex Hull algorithm, collinear points do not

always invalidate a hull edge. Some collinear points may lie on the boundary of the convex hull and should be accepted.

Why is it an error?

- A point with $val == 0$ means it lies on the same straight line as the edge.
- The current code rejects all collinear points without checking whether they lie between the endpoints or outside the edge.
- As a result, the algorithm may reject valid convex hull edges or include incorrect interior edges.
- Proper handling of collinear points is necessary to correctly identify boundary edges.

Correct Fix:

```
if val == 0:
```

```
    if is_between(p1, p, p2):
```

```
        continue    # Accept boundary point
```

After Fixing the Error

```
def hull_edge(p1, p2, points):
```

```
    for p in points:
```

```
        val = orientation(p1, p2, p)
```

```
        # Correct handling of collinear points
```

```
        if val == 0:
```

```
            continue
```

```
        # If any point lies on the opposite side,
```

```
        # the edge cannot be part of the convex hull
```

```
        if val < 0:
```

```
            return False
```

```
    return True
```

```
def hull_edge(p1, p2, points):
    for p in points:

        val = orientation(p1, p2, p)

        # Correct handling of collinear points
        if val == 0:
            continue

        # If any point lies on the opposite side,
        # the edge cannot be part of the convex hull
        if val < 0:
            return False

    return True
```

=== Code Execution Successful ===

Second Error:

for p in points:

 val = orientation(p1, p2, p)

What is the error?

The algorithm performs the orientation calculation for every point, including the endpoints of the candidate edge (p1 and p2). Since these points are already part of the edge being tested, checking their orientation is unnecessary and results in repeated computations.

Why is it an error?

- It performs unnecessary orientation checks for p1 and p2.
- These extra computations reduce the efficiency of the brute force Convex Hull algorithm.
- Skipping the edge endpoints makes the implementation cleaner and slightly more efficient.

Correct Fix

for p in points:

 if p == p1 or p == p2:

 continue

After Fixing the Error

def hull_edge(p1, p2, points):

 for p in points:

 if p == p1 or p == p2:

 continue

 val = orientation(p1, p2, p)

 # Allow collinear boundary points

 if val == 0:

 continue

 # Reject if a point lies on the opposite side

 if val < 0:

```
    return False
```

```
    return True
```

Final Code

```
def orientation(p1, p2, p3):
```

```
    val = (p2[0] - p1[0]) * (p3[1] - p1[1]) - \
          (p2[1] - p1[1]) * (p3[0] - p1[0])
```

```
    if val == 0:
```

```
        return 0
```

```
    elif val > 0:
```

```
        return 1
```

```
    else:
```

```
        return -1
```

```
def hull_edge(p1, p2, points):
```

```
    for p in points:
```

```
        # Skip the endpoints
```

```
        if p == p1 or p == p2:
```

```
            continue
```

```
        val = orientation(p1, p2, p)
```

```
        # Allow collinear boundary points
```

```
        if val == 0:
```

```
            continue
```

```
            if val < 0:
```

```
                return False
```

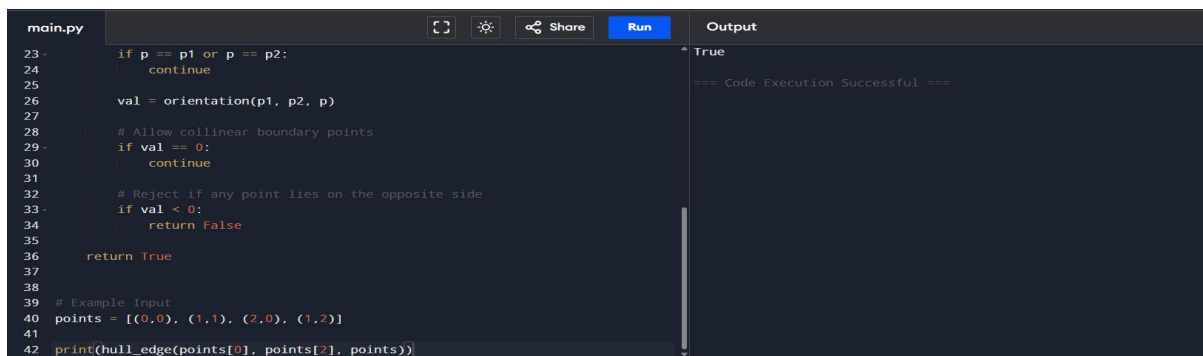
```
    return True
```

```
# Example Input
```

```
points = [(0,0), (1,1), (2,0), (1,2)]
```

```
print(hull_edge(points[0], points[2], points))
```

Output



```
main.py
23-     if p == p1 or p == p2:
24-         continue
25-
26-     val = orientation(p1, p2, p)
27-
28-     # Allow collinear boundary points
29-     if val == 0:
30-         continue
31-
32-     # Reject if any point lies on the opposite side
33-     if val < 0:
34-         return False
35-
36-     return True
37-
38-
39- # Example Input
40 points = [(0,0), (1,1), (2,0), (1,2)]
41
42 print(hull_edge(points[0], points[2], points))
```

Output

```
True
=== Code Execution Successful ===
```

Time Complexity

BestCase:

The best case occurs when all points are already arranged such that every candidate edge can be validated with the minimum number of orientation checks. However, the brute force Convex Hull algorithm still examines every pair of points and compares them with all remaining points. Therefore, the best-case time complexity is $O(n^3)$.

AverageCase:

The average case occurs when the points are randomly distributed. The algorithm checks every pair of points and performs orientation tests with all other points to determine whether the edge belongs to the convex hull. Therefore, the average-case time complexity is $O(n^3)$.

WorstCase:

The worst case occurs when every pair of points must be checked against all other points before determining whether it forms a convex hull edge. Since the algorithm considers all pairs and performs orientation tests for each remaining point, the worst-case time complexity is $O(n^3)$.

Algorithm

1. Start.
2. Input the set of points.
3. Select every possible pair of points (p_1, p_2) as a candidate edge.
4. For each candidate edge, initialize it as a possible convex hull edge.
5. Skip the endpoints (p_1 and p_2) during checking.
6. Compute the orientation of every remaining point with respect to the candidate edge.
7. If a point is collinear, allow it and continue checking the remaining points.
8. If any point lies on the opposite side of the candidate edge, reject that edge.
9. If all remaining points lie on the same side or are collinear, accept the candidate edge as part of the convex hull.
10. Repeat the process for all pairs of points.

11. Display all accepted edges as the Convex Hull.

12. Stop.

Pseudocode

BEGIN

Read the set of points

$n \leftarrow$ number of points

FOR each point i from 0 to $n-1$

 FOR each point j from $i+1$ to $n-1$

$\text{edge} \leftarrow \text{True}$

 FOR each point k from 0 to $n-1$

 IF $k = i$ OR $k = j$ THEN

 CONTINUE

 END IF

$\text{val} \leftarrow \text{Orientation}(\text{points}[i], \text{points}[j], \text{points}[k])$

 IF $\text{val} < 0$ THEN

$\text{edge} \leftarrow \text{False}$

 BREAK

 END IF

 END FOR

 IF $\text{edge} = \text{True}$ THEN

 Display $\text{points}[i]$ and $\text{points}[j]$ as a Convex Hull edge

 END IF

END FOR

END FOR

STOP

END